

基于 ESP32-S3 的 存在感知智能提醒终端 报告

嵌入式系统综合实践

赛道	_____嵌入式比赛乐鑫科技赛道_____
成员姓名	_____王镜熹_____潘程宇_____
学号	_____2452851_____2452853_____

摘要

本项目围绕日常学习和生活场景中的“提醒容易被忽略、无人时提醒无效、人工设置不够自然”等问题，设计并实现了一套基于 ESP32-S3 的存在感知智能提醒终端。系统以 ESP32-S3 DevKitC-1 为核心控制器，集成 OLED 显示、蜂鸣器、LED、LD2410C 毫米波存在感知模块、网页交互界面和 Python 大模型中转服务，实现了手动提醒、网页自然语言提醒、语音输入提醒、到点存在判断、无人暂存提醒、人回来补发提醒以及任务确认完成等功能。

当前版本采用“ESP32-S3 本地状态机 + Python 中转解析”的总体方案：ESP32-S3 负责硬件采集、网页服务、任务管理和提醒执行；Python 服务负责自然语言解析、DeepSeek 调用以及语音识别中转。系统已经完成从基础硬件驱动到自然语言任务创建的主要闭环，并在此基础上规划接入风扇和普通 5V 灯带/COB 灯板，使终端从单一提醒设备扩展为可进行环境反馈和简单语音控制的桌面智能终端。

关键词：ESP32-S3；存在感知；LD2410C；自然语言解析；语音输入；Python 中转；智能提醒终端

1 项目概述

随着智能终端和嵌入式设备在学习、办公和生活场景中的普及，传统闹钟或手机提醒虽然能够在指定时间发出提示，但通常缺少对用户是否在场的判断。如果用户暂时离开座位，提醒可能被错过；如果提醒内容只能通过固定表单输入，使用体验也不够自然。因此，本项目希望将存在感知、自然语言解析和本地提醒执行结合起来，构建一个面向桌面场景的智能提醒终端。

项目最终目标是：用户通过网页或语音输入提醒事项，系统将自然语言解析为结构化任务；到达提醒时间后，ESP32-S3 首先根据 LD2410C 判断附近是否有人；若有人，则通过 OLED、蜂鸣器、LED 以及后续扩展的灯带等执行提醒；若无人，则将任务暂存为待提醒状态，等检测到用户回来后再补发提醒。用户可通过网页按钮或实体按键确认任务完成。

与普通定时提醒相比，本项目的特点在于将“时间触发”和“人在场触发”结合起来，减少无人提醒造成的信息丢失，同时通过 Python 中转服务降低 ESP32-S3 端的算法复杂度，使大模型解析和语音识别可以在电脑端或服务器端完成。

1.1 设计目标

1. 实现稳定的 ESP32-S3 硬件控制，包括 OLED、LED、蜂鸣器、按键和人体存在传感器。
2. 实现网页端提醒任务添加，支持 HH:MM 手动输入和自然语言输入。
3. 实现 Python 中转服务，使自然语言先由本地规则解析，规则失败时再调用 DeepSeek 大模型。
4. 实现语音输入入口，使语音识别结果复用现有 /parse_text 解析流程。
5. 实现存在感知提醒状态机：WAITING、PENDING、ALERTING、DONE。
6. 扩展风扇和普通 5V 灯带/COB 灯板控制，实现简单的语音开关和提醒联动。

1.2 应用场景

- 学习桌面提醒：如喝水、开会、提交作业、复习计划等。
- 宿舍或实验室桌面终端：在用户离开时不盲目提醒，回来后补发。
- 智能交互演示：通过网页语音或 AI 输入自然语言，展示从语言理解到硬件执行的闭环。
- 环境反馈控制：后续可通过风扇、台灯、RGB 灯环、语音播报等提升终端表现力。

2 系统总体设计

系统采用分层设计。ESP32-S3 作为现场控制层，负责传感器读取、网页服务、任务状态机和执行设备控制；Python 服务作为智能解析层，负责自然语言解析、DeepSeek 调用和可选的语音识别；用户通过手机或电脑网页完成输入和状态查看。

层级	组成	主要作用
用户交互层	网页表单、网页语音输入、实体确认按键	添加提醒任务、语音控制外设、确认提醒完成
智能解析层	Python Flask 服务、规则解析、DeepSeek 接口、语音识别接口	将自然语言或语音转文字解析为结构化 JSON
控制执行层	ESP32-S3、WebServer、任务状态机、HTTPClient	保存任务、判断状态、调用 Python、控制外设
感知与输出层	LD2410C、OLED、蜂鸣器、LED、风扇、灯带	检测是否有人并通过多种方式执行提醒或反馈

表 1 系统分层结构

2.1 系统工作流程

系统的典型工作流程如下：用户在网页中通过手动表单、AI 输入框或语音输入提交提醒内容；ESP32-S3 将自然语言文本发送到 Python 服务的 /parse_text 接口；Python 先尝试本地规则解析，若规则无法处理复杂表达，则调用 DeepSeek 大模型进行兜底解析；解析成功后返回任务标题、延迟秒数或目标时间；ESP32-S3 创建当前任务并进入 WAITING 状态；到达提醒时间后，系统读取 LD2410C 的人体存在结果，若有人则进入 ALERTING 状态，若无人则进入 PENDING 状态；用户回来后由 PENDING 转为 ALERTING；确认完成后进入 DONE。

步骤	阶段	说明
1	输入	用户通过网页、AI 文本或语音输入提醒内容
2	解析	Python 返回结构化 JSON，例如 title、delay_seconds、type
3	创建任务	ESP32-S3 保存任务并等待触发时间
4	存在判断	到点后读取 LD2410C OUT，判断是否有人
5	提醒执行	有人则 OLED/蜂鸣器/LED/灯带提醒，无人则暂存
6	确认完成	网页按钮或 GPIO6 实体按键确认，任务进入 DONE

表 2 系统主要工作流程

2.2 硬件组成

硬件模块	型号或说明	当前作用
主控	ESP32-S3 DevKitC-1 1U-N8R8	负责网页服务、GPIO 控制、任务状态机和网络通信
显示模块	0.96 寸 I2C OLED，SSD1306，128×64	显示当前时间、任务状态和提醒内容
存在感知	LD2410C 毫米波雷达模块	通过 OUT 输出判断是否有人
提示设备	有源蜂鸣器、外接 LED	提醒时发声和闪烁
输入设备	GPIO6 实体按键	用户确认任务完成
语音输入	网页语音输入；可选 INMP441 硬件麦克风	将语音转换为自然语言文本
执行扩展	LR7843 MOSFET、5V 风扇、5V COB 灯板	实现风扇和灯带开关控制
音频输出拓展	MAX98357A I2S 功放 + 小喇叭（规划）	后续实现语音播报或提示音输出

表 3 硬件组成

2.3 软件组成

软件部分	技术/库	功能
ESP32 固件	Arduino IDE、WiFi.h、WebServer.h、HTTPClient.h、ArduinoJson.h、time.h	联网、网页服务、NTP 时间、HTTP 请求、JSON 解析
OLED 驱动	Wire.h、Adafruit_GFX、Adafruit_SSD1306	I2C 显示任务和状态信息
Python 中转服务	Flask、requests、python-dotenv	提供 /parse_text、/voice、/health 等接口
大模型解析	DeepSeek API	处理复杂自然语言提醒表达
语音识别	网页语音输入；可选 faster-whisper	将语音转换为文字后复用解析接口

表 4 软件组成

3 已完善功能

当前项目已经完成了从基础硬件驱动到自然语言提醒创建的主要闭环。各阶段功能并非一次性完成，而是按“基础外设测试—联网与时间—存在感知—网页输入—自然语言解析—语音输入—外设扩展”的顺序逐步集成。

序号	功能模块	完成情况	说明
1	串口、LED、蜂鸣器测试	已完成	串口输出正常，LED 和有源蜂鸣器可由 GPIO 控制
2	OLED 显示	已完成	I2C OLED 使用 GPIO8/GPIO9，能够显示

			任务与状态信息
3	Wi-Fi 与 NTP 时间	已完成	ESP32-S3 能联网并获取北京时间，为定时提醒提供基础
4	LD2410C 存在感知	已完成并持续优化	OUT 接 GPIO15，当前建议使用 5V 供电并加入稳定滤波
5	网页添加任务	已完成	手机可访问 ESP32 网页并添加任务
6	状态机提醒逻辑	已完成	实现 WAITING/PENDING/ALERTING/DONE 四态逻辑
7	Python 中转解析	已完成	本地规则优先，规则失败时调用 DeepSeek
8	自然语言创建任务	已完成	支持“几分钟后”“晚上八点”“明天晚上八点”等表达
9	语音输入	已完成	语音转文字后复用 /parse_text 接口创建任务
10	风扇与灯带控制	正在接入/扩展	通过 LR7843 + GPIO13/GPIO14 实现开关和语音控制

表 5 已完善功能汇总

3.1 已测试的自然语言输入

- “五分钟后提醒我喝水”：由本地规则解析为延迟 300 秒的提醒任务。
- “晚上八点提醒我开会”：根据当前日期和时间换算目标提醒时间。
- “明天晚上八点提醒我交作业”：识别日期偏移和中文时间表达。
- “我两分钟后有个会议，提前一分钟提醒我”：识别事件时间和提前提醒时间，最终按 1 分钟后触发。

3.2 语音输入实现效果

语音输入功能采用轻量化路线，避免一开始就在 ESP32-S3 端运行本地语音识别模型。当前优先使用手机或电脑网页端语音识别，将识别到的文字填入 AI 输入框，再通过原有 /parse_text 接口完成解析。这样做能够最大程度复用已有自然语言解析流程，减少对 ESP32-S3 端内存和算力的压力，同时提高比赛演示的稳定性。

在硬件麦克风方案中，INMP441 可通过 I2S 接入 ESP32-S3，ESP32 录制 WAV 后上传 Python /voice 接口，由 Python 侧 faster-whisper 识别。该方案作为拓展功能保留，但调试复杂度较高，因此当前课程展示优先采用网页语音输入作为主线。

4 关键实现方法

本项目的关键并不在单个外设驱动，而在于将多个模块整合成稳定的“输入—解析—判断—提醒—确认”闭环。下面从状态机、Python 中转、语音输入和硬件控制四个方面说明实现方法。

4.1 存在感知提醒状态机

状态	含义	进入条件	退出条件
WAITING	任务已创建，等待提醒时间到达	成功添加提醒任务后进入	到达提醒时间后根据是否有人转入 ALERTING 或 PENDING
PENDING	到点时无人，暂存待提醒	到点但 LD2410C 判断无人	检测到人回来后进入 ALERTING
ALERTING	正在提醒用户	到点有人或 PENDING 后检测到人	用户按键或网页确认后进入 DONE
DONE	任务已完成	用户确认完成	新任务创建后重新进入 WAITING

表 6 任务状态机设计

状态机设计使系统具备了存在感知能力。普通提醒设备只在时间到达时提醒，而本系统在时间触发后还要判断用户是否在场。如果无人，提醒不会被简单丢弃，而是进入 PENDING 状态，等人回来后再提醒。该逻辑能够更好地适应宿舍、实验室、桌面学习等实际场景。

4.2 Python 中转与自然语言解析

Python 中转服务使用 Flask 实现，主要提供 /parse_text 接口。ESP32-S3 通过 HTTP POST 将用户输入的自然语言发送给 Python 服务，Python 返回结构化 JSON。解析逻辑采用“本地规则优先、大模型兜底”的方式：常见表达由规则直接解析，速度快且稳定；对于“周五晚上十点前交作业，提前一小时提醒我”等复杂表达，再调用 DeepSeek 大模型处理。

这种设计使 ESP32-S3 不需要在 Arduino 代码中处理复杂中文语义，也不需要保存大模型密钥和长提示词，从而降低了固件复杂度。ESP32 只需要理解 Python 返回的简单 JSON 字段，例如 success、title、delay_seconds、remind_time 或 control action。

字段	示例	含义
success	true	解析是否成功
type	reminder / control	区分提醒任务和外设控制命令
title	喝水 / 打开风扇	任务标题或控制动作标题
delay_seconds	300	相对当前时间的延迟秒数
remind_time	20:00	仅有 HH:MM 时使用的时间字段
action	fan_on / lamp_off	控制类命令的具体动作

表 7 Python 返回 JSON 的主要字段

4.3 语音输入链路

当前语音输入的主线流程为：用户点击网页语音输入按钮，浏览器将语音转换为文字，文字自动填入 AI 输入框，用户确认后提交给 ESP32；ESP32 将文本转发给 Python /parse_text；Python 完成解析后返回 JSON；ESP32 根据 JSON 创建任务或执行风扇、灯带等控制动作。

该方案的优势是轻量、稳定、可演示，不需要 ESP32-S3 长时间持续监听，也不需要本地运行唤醒词模型。硬件麦克风方案可以作为后续优化，但当前版本不建议默认持续监听，以免影响网页响应、任务状态机和系统稳定性。

4.4 风扇与灯带控制实现

风扇和普通 5V 灯带/COB 灯板均属于电流较大的外部负载，不能直接接 ESP32 GPIO。项目使用 LR7843 MOSFET 模块作为电子开关。ESP32 的 GPIO13 控制风扇模块 PWM，GPIO14 控制灯带模块 PWM；风扇和灯板的正极接 5V，负极通过 LR7843 的 LOAD 端受控接地。

软件侧可新增 setFan()、setLamp() 等函数，并在网页上加入 Fan ON/OFF、Lamp ON/OFF 按钮。同时，在 Python 的本地规则中加入“打开风扇”“关闭灯带”等控制命令解析，返回 type=control 和 action 字段。ESP32 收到 control 类型 JSON 后直接执行控制，不创建提醒任务；收到 reminder 类型 JSON 时继续按原流程创建提醒。

5 硬件连接与扩展设计

本节整理当前主要硬件连接，便于课程答辩、代码维护和后续 PCB 设计。实际接线时应以开发板丝印和模块丝印为准，尤其是电源正负极、MOSFET 模块的 LOAD 端以及可编程 RGB 灯环的 DI/DO 方向。

模块/信号	ESP32-S3 引脚或电源	说明
OLED SDA	GPIO8	I2C 数据线
OLED SCL/SCK	GPIO9	I2C 时钟线
有源蜂鸣器	GPIO4	提醒声音输出
外接 LED	GPIO5	提醒灯光输出
确认按键	GPIO6 与 GND	按下时确认任务完成
LD2410C OUT	GPIO15	人体存在数字输出
LD2410C VCC	5V	建议使用 5V 供电，GND 共地
INMP441 SCK	GPIO10	I2S 麦克风时钟，若保留硬件语音
INMP441 WS	GPIO11	I2S 字选择信号
INMP441 SD	GPIO12	I2S 麦克风数据输入
风扇 LR7843 PWM	GPIO13	控制 5V 风扇开关
灯带 LR7843 PWM	GPIO14	控制普通 5V COB 灯板开关

表 8 当前引脚连接表

5.1 风扇接线

风扇通过一块 LR7843 模块控制。模块大电流端丝印通常为“- / LOAD / +”，控制端为“GND / PWM”。接线方式为：模块“+”同时接 ESP32 5V 和风扇红线，模块 LOAD 接风扇黑线，模块“-”接 ESP32 GND；控制端 PWM 接 GPIO13，控制端 GND 接 ESP32 GND。

5.2 普通 5V 灯带/COB 灯板接线

普通 5V 灯带或 COB 灯板同样通过另一块 LR7843 模块控制。灯板正极焊红线，负极焊黑线；模块“+”同时接 ESP32 5V 和灯板红线，模块 LOAD 接灯板黑线，模块“-”接 ESP32 GND；控制端 PWM 接 GPIO14，GND 接 ESP32 GND。该接法本质是低端开关控制，即让负极通过 MOSFET 受控导通。

5.3 供电注意事项

当前调试阶段可暂时使用 ESP32 开发板 5V 给风扇和灯板供电，但风扇、灯板、LD2410C、蜂鸣器等外设同时工作时，USB 口供电可能不足，导致 ESP32 重启、传感器输出不稳定或模块发热。正式演示建议使用 5V 2A 以上电源或 Type-C 供电板，并保证外部电源 GND 与 ESP32 GND 共地。

6 测试流程与结果分析

为避免多个模块同时接入造成排查困难，项目采用分阶段测试方法。每一项功能先用最小程序或单独接口验证，再并入主程序。

测试项	测试方法	预期结果
OLED 显示	运行 SSD1306 初始化与显示测试	屏幕正常显示文字
Wi-Fi 与 NTP	连接热点或路由器，打印当前时间	串口显示联网成功并获取时间
LD2410C	串口打印 GPIO15 RAW 和稳定状态	有人时稳定为有人，无人时延迟转为无人
网页手动提醒	输入 HH:MM 和内容	到点后进入提醒流程
自然语言提醒	输入“五分钟后提醒我喝水”	Python 返回 JSON，ESP32 创建任务
语音输入	网页点击语音输入并提交	识别文本进入 AI 解析流程
风扇控制	网页点击 Fan ON/OFF 或输入“打开风扇”	GPIO13 控制 LR7843，风扇开关
灯带控制	网页点击 Lamp ON/OFF 或输入“关闭灯带”	GPIO14 控制 LR7843，灯板开关
提醒联动	设置短时间提醒并保持人在场	OLED、蜂鸣器、LED、灯带等联动提醒

表 9 测试流程与预期结果

测试表明，项目的核心提醒流程已经能够完成从输入到提醒执行的闭环。自然语言解析对常见提醒表达具有较好稳定性，语音输入通过网页端识别后能够复用同一解析接口。后续测试重点应放在存在感知稳定性、外部负载供电稳定性和风扇/灯带联动逻辑上。

7 存在问题与优化方向

虽然项目已经完成主要功能，但从课程设计向更成熟的作品过渡，还需要在稳定性、交互体验、硬件结构和软件架构方面继续优化。

7.1 存在感知稳定性优化

LD2410C 工作时会有发热，轻微温热属于正常现象，但若发烫明显或 OUT 输出不稳定，需要优先检查供电、接线和安装位置。当前建议使用 5V 供电，GPIO15 只读取 OUT 信号，并在软件中加入滤波逻辑：连续多次检测到有人才确认有人，连续若干秒无人再确认无人。这样可以减少 OUT 抖动对状态机的影响。

7.2 供电与驱动优化

风扇和灯板接入后，系统电流需求明显上升。若继续使用电脑 USB 口给所有模块供电，可能出现 ESP32 重启、传感器误触发、灯光闪烁等问题。后续应使用独立 5V 2A 或更高规格电源，并设计统一电源分配板，确保 ESP32、LD2410C、LR7843、风扇、灯板和音频模块共地。

7.3 任务管理优化

当前系统仍为单任务版，新任务会覆盖旧任务。后续可设计多任务列表，每个任务包含 ID、标题、目标时间、状态和优先级，并支持网页查看、删除、完成单个任务。同时可以将任务保存到 ESP32 的 NVS 或 SPIFFS，断电后仍能恢复未完成提醒。

7.4 语音交互优化

当前版本不建议加入真正的持续唤醒词，因为这会显著增加 I2S 采样、唤醒检测、误触发控制和功耗管理的复杂度。更合理的路线是先保持网页按钮触发语音输入，后续再扩展本地唤醒词或物理按键唤醒。硬件麦克风方案可以继续作为加分项，但不应影响主流程稳定性。

7.5 交互与外观优化

- 网页端增加更清晰的任务卡片、当前状态显示、控制开关和错误提示。
- OLED 界面区分待提醒、提醒中、已完成和联网状态。
- 增加灯带亮度控制或呼吸效果，使提醒更直观。
- 设计外壳和固定结构，将 LD2410C、OLED、灯带、风扇进行合理布局。
- 后续绘制 PCB，减少杜邦线造成的接触不良和调试混乱。

7.7 下一阶段功能规划

需要说明的是，当前项目已经完成了从网页/语音输入到存在感知提醒执行的初步闭环，但整体仍处于课程设计原型的初步阶段。现阶段重点验证了 ESP32-S3、Python 中转解析、LD2410C 人体存在检测、OLED 显示、蜂鸣器、LED、风扇和普通 5V 灯板等模块之间的基本协同关系，后续仍需要围绕环境感知、自动控制、外观结构和工程化实现继续完善。

下一步可加入温湿度传感器，用于实时采集桌面环境温度和湿度。当系统检测到有人在场且环境温度较高时，可自动开启风扇或提高风扇运行强度；当无人时则保持关闭或低功耗状态，从而使风扇不再只是手动开关设备，而是能够根据“是否有人 + 环境温度”进行联动控制。

同时可加入光照传感器，用于判断当前桌面环境亮度。系统可在 LD2410C 检测到有人在场且环境光较暗时自动开启台灯或灯板，并根据光照强度调节灯光亮度；当环境光充足或用户离开时自动关闭照明。这样可以形成“人在场检测 + 环境信息采集 + 自动执行设备控制”的智能桌面场景。

在外设控制方面，风扇和普通 5V 灯板可继续通过 LR7843 MOSFET 模块驱动，后续代码中可将“打开风扇”“关闭灯带”等语音控制命令扩展为“温度高一点自动开风扇”“光线太暗打开台灯”等更自然的复合指令。Python 中转服务负责理解语义并返回结构化控制指令，ESP32-S3 端只负责执行对应 GPIO 控制和状态显示。

在硬件结构方面，后续需要进行外观优化和集成化设计。当前原型仍以杜邦线、模块和临时焊接为主，容易出现接触不良、线缆杂乱和供电不稳定等问题。下一阶段应设计固定支架或外壳，将 OLED、LD2410C、灯板、风扇、按键和电源接口进行合理布局，使设备更像一个完整的桌面智能终端。

在工程实现方面，后续可进一步绘制 PCB，将 ESP32-S3 主控、传感器接口、MOSFET 驱动、电源分配、按键、OLED 接口和外设端子统一规划，减少飞线和面包板连接。PCB 设计时应重点考虑 5V 供电能力、3.3V 逻辑电平、GND 共地、风扇和灯板负载电流、接口防反接以及后期维护便利性。

因此，项目后续发展方向并不是单纯增加外设数量，而是把已经完成的“提醒终端”进一步升级为具备存在感知、环境感知和自动执行能力的集成式桌面语音助手。

8 总结

本课程项目完成了一套基于 ESP32-S3 的存在感知智能提醒终端初步原型。项目从基础外设测试入手，逐步实现 OLED 显示、Wi-Fi 联网、NTP 时间同步、LD2410C 存在感知、网页添加任务、自然语言解析、Python 大模型中转、语音输入以及风扇和普通 5V 灯板控制等功能，形成了较完整的课程设计阶段闭环。

当前系统已经能够支持用户用自然语言或语音输入提醒任务，并在到点时结合人体存在状态决定立即提醒或延迟补发，体现了嵌入式硬件控制和人工智能解析能力的结合。但从完整产品角度看，项目仍处于初步阶段，后续应继续加入温湿度传感器、光照传感器等环境感知模块，并配合 LD2410C 实现有人时自动采集环境信息、自动开关风扇、调节台灯亮度等功能。同时，还需要进一步优化外观结构、设计 PCB、规范焊接与供电，使系统从课程实验原型逐步发展为更稳定、更完整的集成式桌面智能终端。

•